Data Miner · entregables, Excel de prueba y evaluación de VeriMine

Challenge 06 · BoxLang + AI Hackathon El Salvador · 11/10/2026

En una página

• **Ya hay 10 entregables listos** (web, equipo multiagente, .bat, módulo, MCP, informe, capturas, 3 PDF, datos de prueba) y el sistema produce **15 tipos de salida** (sección 1 y 2).

• **Nuevo hoy:** 4 Excel sucios y desnormalizados de una distribuidora salvadoreña ficticia; un **detector de normalización** (1FN, 2FN, 3FN, anomalías, nombres escritos de varias formas) que además crea las tablas normalizadas; y la parte barata de VeriMine: **cifra contra SQL** y **bloques sellados**. Todo probado en BoxLang 1.18 con los Excel nuevos.

• **VeriMine encaja y conviene hacerlo**, pero en otro orden: primero comparar la cifra de cada hecho con su SQL (hoy no se hace), luego los bloques sellados, y el consenso entre agentes solo si sobra tiempo.

• **Falta:** integrarlo en tu copia del PC (pasos en codigo/INTEGRAR.md) y probar con un LLM real.

1. Entregables disponibles

Entregable	Dónde	Estado
App web: subir archivos, diagnóstico inmediato, misiones en lenguaje natural, informe	www/ (iniciar-web.bat)	✓ en modo demo
Equipo multiagente: coordinador + 3 agentes (compacto) o + 7 (completo), pipeline con structured output	models/Equipo.bx	✓ con mock · ✗ sin probar con LLM real
Instalación y arranque con doble clic	revisar-requisitos, instalar, config, iniciar-web, demo-consola, iniciar-mcp (.bat)	✓
Módulo reutilizable para la comunidad	modulo/bx-datatools/	✓ con ejemplo sin API key
Base de datos vía MCP (solo lectura)	mcp/ + ConectorMCP.bx	✓ real con mcp-server-sqlite
Informe HTML de ejemplo (offline, imprimible a PDF)	informe-demo.html	✓
Capturas para la presentación (7)	captura-*.png, rediseno/	✓
Documentos: propuesta, pasos y requerimientos, resumen de challenges	boxlang/*.pdf	✓
Datos de prueba: crédito, precios, clientes SV, operaciones SV, archivos sucios	data/, pruebas/fuentes/	✓
Nuevo: 4 Excel sucios, Normalizacion.bx, Verificador.bx, 2 scripts de prueba, guía de integración	boxlang/entregables/	✓ probados · sin integrar

2. Salidas posibles del sistema

Salida	Cuándo sale	Forma
Diagnóstico de calidad: puntaje 0-100, problemas con severidad y recomendación	Al subir un archivo, sin IA	Tarjeta en la web / JSON
Aviso de anonimización (candado): qué columnas y cómo	Al subir	Web e informe
Respuesta de la misión: respuesta, hechos con su SQL (✓/✗), interpretaciones, siguientes preguntas	Cada pregunta	Web y JSON
Traza de lo que hizo cada agente (bitácora de evidencia)	Cada misión	Web
Gráficas: barras, línea, pastel, dispersión, mapa de calor	Analista / Estadístico	Chart.js en web e informe

Salida	Cuándo sale	Forma
Informe HTML descargable, sin internet	Al final de la misión	HTML (→ PDF desde el navegador)
Tablas nuevas consultables: _limpia, pronostico_stock, prioridad_ventas, entrenamiento_*	Tras limpiar, pronosticar o priorizar	Tablas SQL
Esquema SQL propuesto (CREATE TABLE con tipos, NOT NULL, PK, CHECK)	proponer_esquema	Texto DDL
Fuga de datos para ML y consejos de recolección	Ingeniero de datos	Lista priorizada
Estadística: descriptiva, correlación, covarianza; finanzas: retorno, volatilidad, Sharpe, drawdown, VaR	Estadístico	Tabla + gráfica
ML: métricas contra línea base, variables importantes, predicciones	Científico ML (Tribuo)	Tabla + gráfica
Operaciones: cuándo se agota cada SKU y cuánto pedir; ventas a priorizar	Analista / Científico ML	Tabla + hechos verificables
Consultas a la base remota por MCP	Si hay MCP_DB_URL	Hechos con origen db
Nuevo: nivel de normalización, clave, entidades, % de celdas repetidas, anomalías, tablas partidas, DDL y tablas _n_	revisar_normalizacion, normalizar_tabla	JSON + tablas SQL
Nuevo: cifra contrastada (✓ / ⚠) y cadena: «4 bloques · 1 verificado · 3 disputa · cadena íntegra»	Cada hecho	Web e informe

3. Los 4 Excel de prueba

Distribuidora Cuscatlán, S.A. de C.V. (ficticia): 9 sucursales de San Salvador a Usulután, 18 productos de consumo, 6 proveedores, clientes consumidor final y contribuyentes (Factura / CCF, IVA 13%), pagos en efectivo, tarjeta, transferencia y Bitcoin. Nombres, DUI, NIT, NRC y teléfonos son inventados. Se regeneran iguales con excel/generar_excel_sucios.py.

Archivo	Hojas	Lo sucio a propósito	Lo que el sistema encontró
ventas_sucursales_2026.xlsx	Ventas Jul-Sep (1,344 líneas), Octubre (parcial), Resumen por sucursal	Todo en una tabla (factura + cliente + sucursal + producto + proveedor); título arriba; fila TOTAL GENERAL; 28 líneas duplicadas; fechas en 4 formatos; precios como 4.25, $4.25 y 4,25; sucursal escrita de 3-4 formas; el gerente de Santa Ana cambia a medias; el café sube de precio a medias; teléfono viejo del proveedor; categorías en conflicto; devoluciones negativas; fórmulas; meses en columnas	Nivel 1FN, clave Factura + SKU, 59% de celdas repetidas, 7 tablas propuestas, 4 anomalías (gerente, precio, categoría, teléfono), variantes de sucursal, fila de totales, octubre = misma tabla partida, meses como columnas
sucursales_empleados.xlsx	Sucursales, Planilla (56)	Ubicación «dirección, municipio, departamento» en una celda; empleados y teléfonos como lista; Meta Q1..Q4; código SA01 repetido con otro gerente; salarios $1,200.00 / 1200,00 / US$ 1200; jefe con dos teléfonos	No cumple 1FN: atributo compuesto, 3 listas, grupo repetido; código repetido con datos distintos; en planilla Cargo → Salario y variantes de sucursal
productos_inventario.xlsx	Catálogo e inventario, Proveedores	Proveedor (NIT, teléfono, contacto) copiado en cada producto; stock en 9 columnas (una por sucursal) con N/D y negativos; SKU repetido con otro precio; presentaciones como lista	No cumple 1FN; Proveedor = NIT = teléfono = contacto (3FN); SKU repetido con datos distintos; sin clave clara
pedidos_mayoristas.xlsx	Pedidos (120)	Producto 1..5, Cant 1..5, Precio 1..5 en columnas; datos del cliente repetidos en cada pedido; estados con mayúsculas mezcladas	Grupo repetido Producto/Cant/Precio × 5 (40% de celdas vacías); Cliente = NRC determina giro y departamento (3FN)

La salida completa está en codigo/salida-probar-normalizacion.txt. Ejemplo real de lo que ve el usuario:

```
[Anomalía] Gerente_sucursal: SA01 → 'Mauricio Linares' (127) vs 'Verónica Sandoval' (35)
[Anomalía] Precio_unitario: CAF-001 → '4,25' (59) vs '4.5' (7) · LAC-001 → '3.8' (57) vs '3.5' (6)
[Variantes] Sucursal: SS01: 'S.S. Centro' / 'san salvador centro' / 'SAN SALVADOR CENTRO' / 'San Salvador
Centro'
[2FN] No_Factura determina Fecha, Tipo_DTE, Vendedor, Cliente, Forma_de_pago. Se repite en 796 filas
```

4. Detección de normalización (models/Normalizacion.bx)

Es determinista: no usa el LLM y no toca la tabla original. Tres tools nuevas para el agente Datos: **revisar_normalizacion**, **normalizar_tabla** y **tablas_partidas**.

- **1FN:** celdas con listas («Ana, José, Luis»), atributos compuestos («dirección, municipio, departamento»), grupos repetidos de columnas (Producto_1..5, Meta_Q1..Q4, Stock_SS01..US01, Julio/Agosto/Septiembre).
- **2FN y 3FN:** busca dependencias funcionales aproximadas (A determina B en al menos 90% de las filas y mejor que adivinar la moda). Junta las que son el mismo dato (SKU = Producto), quita las transitivas y detecta la clave (una columna o un par). Si la columna que determina es parte de la clave, es 2FN; si no, 3FN.
- **Anomalías de actualización:** un mismo código con dos valores (precio, gerente, teléfono, categoría). Dice cuántas filas tiene cada versión, para decidir si es error o un cambio en el tiempo.
- **Nombres escritos de varias formas:** «S.S. Centro» y «San Salvador Centro» para SS01. Es el caso «Norte / norte» del PDF de VeriMine, resuelto antes de llegar al SQL.
- **Extra:** fila de totales dentro de los datos, código que debería ser único y se repite, hojas que son la misma tabla partida, % de celdas copiadas, DDL con PRIMARY KEY y REFERENCES.
- **normalizar_tabla** crea las tablas _n_ en el catálogo. Con ventas: 8 tablas; el SELECT con JOIN reconstruye las 1,315 líneas (1,344 menos 28 duplicadas y la fila de totales) y un JOIN de 5 tablas da las ventas por departamento.

Límites

- Son reglas: con menos de 20 filas solo revisa 1FN y claves repetidas. Puede reportar una dependencia que hoy se cumple pero no es regla del negocio (Cargo → Salario). El agente debe presentarlas como «parece», no como hecho.
- Factura con formatos mezclados (F-10450 y 10450) queda como dos facturas: 548 en vez de ~520. Conviene limpiar antes (limpiar_tabla) y luego normalizar.
- No interpreta encabezados en dos filas ni celdas combinadas (el lector de Excel actual tampoco).
- Tiempo: unos 6-7 s para 1,344 filas × 28 columnas la primera vez; las tablas chicas, menos de 0.2 s.

5. Evaluación de VeriMine

Veredicto: sí, encaja con Data Miner y le da al pitch una historia de confianza completa. El diagnóstico del compañero es correcto, pero el problema actual es más básico de lo que dice el PDF, y eso cambia el orden: **(1)** comparar la cifra con el SQL, 0 llamadas al LLM, ya hecho; **(2)** bloques sellados, 0 llamadas, ya hecho; **(3)** consenso entre agentes, solo si sobra tiempo.

Lo que el PDF tiene bien

- Re-ejecutar el mismo SQL no detecta un SQL mal planteado: un filtro incompleto da el mismo número equivocado dos veces y sale ✓.
- Limitar el consenso a los 5 hechos más importantes y al modelo ligero es la forma correcta de cuidar la cuota.
- Empezar por los bloques sellados porque no gastan cuota y funcionan solos.

Lo que hay que corregir

- **Hoy la verificación es más débil de lo que asume el PDF.** Equipo.bx, función verificar() (líneas 265-270), solo revisa que el SQL corra y traiga al menos una fila. No compara el número de la afirmación con el resultado. Un hecho que dice «los pedidos suman $250,000» con un SQL que da $20,655.30 sale ✓. **Verificador.contrastar()** lo arregla sin LLM: compara cada cifra con ±1%, respeta el redondeo y entiende que 35% es 0.35. En la prueba, 3 de 4 hechos que hoy salen ✓ quedan en ⚠ disputa.
- **Sellar los datos, no solo la afirmación.** La demo del PDF es «altero un número a mano y la cadena se rompe». Si el bloque solo guarda el texto y el SQL, cambiar un dato de la tabla no rompe nada. Verificador guarda también el hash del resultado y una huella de cada tabla que usa el SQL; al cambiar un monto en la tabla, la cadena sale rota desde el bloque 1.
- **Los decimales rompen la cadena solos.** Un SUM de decimales da 20655.300000000007 en una corrida y 20655.30000000001 en otra según el orden de suma. En la prueba eso rompía la cadena sin que nadie tocara nada. Verificador redondea a 6 decimales antes del hash.
- **El consenso no arregla datos sucios.** Si la tabla tiene «Norte» y «norte», el validador puede cometer el mismo error que el analista y salir ✓✓. La normalización y la limpieza atacan la causa; el validador debería consultar la tabla _limpia o _n_.
- **El validador necesita el esquema.** Si solo recibe la afirmación, falla por nombres de columna, no por desacuerdo real. Dale listar_tablas y perfilar_tabla, y muestra en la UI los dos SQL lado a lado cuando haya disputa.
- **Costo del consenso:** 5 hechos × 2-3 llamadas = 10-15 llamadas más. La misión de la web hoy usa 15, así que casi la duplica. Con plan gratuito: máximo 3 hechos, solo los que tienen cifra, y otro modelo distinto al del analista (si es el mismo modelo, comparte sus sesgos).

Cómo contarlo

- La analogía con Bitcoin atrae, pero un jurado técnico puede preguntar dónde está la red o el consenso distribuido. Es una cadena de hashes, como git o un libro contable sellado: **detecta** cambios, no los impide, y quien controla el servidor podría recalcularla. Dicho así es honesto y sigue siendo fuerte.
- Mantener «Don't trust. Verify.» como frase. La historia completa son 4 capas de confianza: **anonimizar al entrar → calidad y normalización → verificar la cifra → sellar**. El informe cierra con «12 bloques · 9 consenso · 2 verificados · 1 disputa · cadena íntegra».
- La cadena vive en la memoria de la sesión. Para la demo alcanza; en producción se guarda en un archivo o base y se firma.

Demo de 60 segundos

1) Subir ventas_sucursales_2026.xlsx → tarjeta: «59% de celdas repetidas, el precio del café tiene dos valores, SS01 escrito de 4 formas». 2) Preguntar «¿cuánto vendió cada departamento?» → hechos ✓ con su SQL y uno ⚠ si la cifra no cuadra. 3) Botón «Verificar cadena» → íntegra. 4) Cambiar un monto en la tabla y volver a verificar → rota desde el bloque 1.

6. Hallazgos laterales al probar

- **Bug en Catalogo.numeroTexto():** convierte 1.35 en 1.350000000000000088817841970012523233890533447265625. Se ve en columnas mixtas (precio con $ en unas filas y número en otras). El arreglo es una línea (INTEGRAR.md, punto 5).
- **La anonimización deja nombres a la vista** en columnas llamadas Cliente, Vendedor, Gerente_sucursal, Gerente y Jefe_inmediato, y en la lista de Empleados. Esos nombres llegarían al LLM. Conviene sumar esos nombres de columna a la regla de «nombre de persona». Al revés, AFP (Confía, Crecer) se trata como documento fiscal y se pseudonimiza sin necesidad.

7. Qué sigue

Paso	Quién	Tiempo estimado
Copiar Normalizacion.bx, Verificador.bx, pruebas y Excel; conectar las tools y verificar() (codigo/INTEGRAR.md)	Hilo «Data Miner en carpeta nueva» en tu PC	45-60 min
Botón «Verificar cadena» y mostrar ⚠ y la cadena en el informe	Mismo hilo	~30 min
Probar la misión con un LLM real y los Excel nuevos	Mario	~20 min
Consenso entre agentes (validador independiente)	Solo si sobra tiempo	~2 h según el PDF

Archivos: boxlang/entregables/excel/ (4 Excel + generador), boxlang/entregables/codigo/ (models/Normalizacion.bx, models/Verificador.bx, probar-normalizacion.bxs, probar-verificador.bxs, salidas reales, INTEGRAR.md).